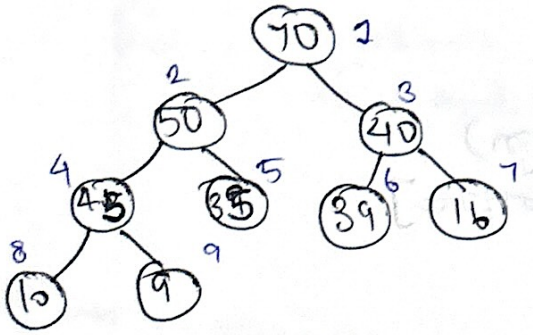
(1x) Heap → Special type of complete binary tree where every level is filled completely and we fill from left side. (leaf node)

Max Heap

for every node i, the value of node is less than or equal to its parent value

$A[Parent(i)] \geq A[i]$

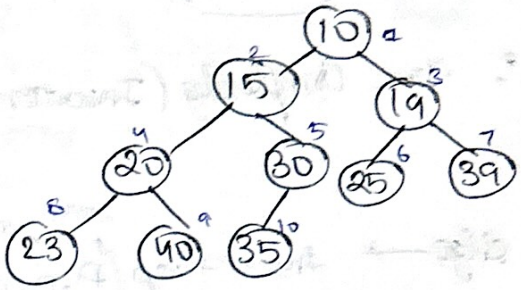


	70	50	40	45	35	39	16	10	9

Min Heap

for every node i, the value of node is greater than or equal to parent node.

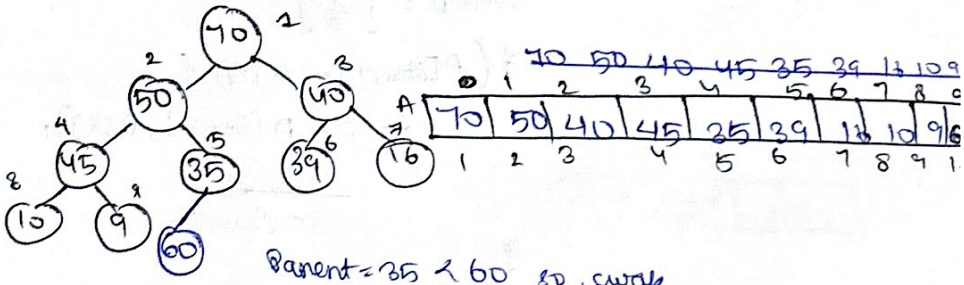
$A[Parent[i]] \leq A[i]$



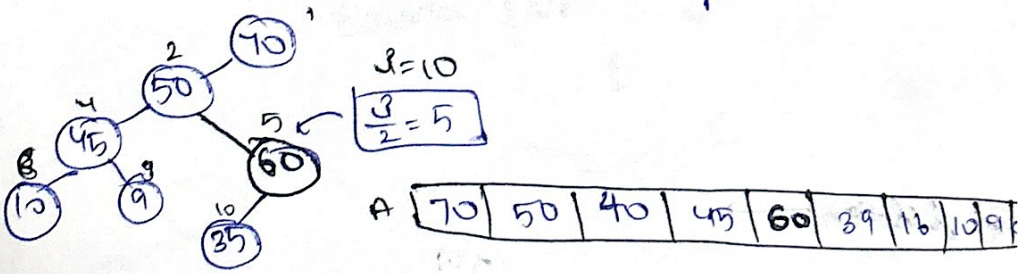
	10	15	19	20	30	25	39	23	40	35

(*) Insertion in Max-heap ⇒

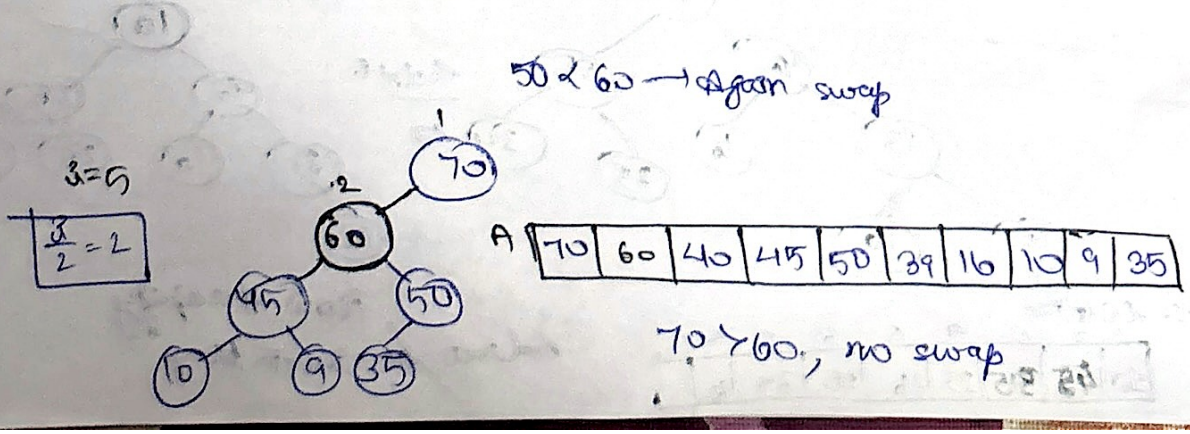
① Insert 60.



Parent = 35 < 60 so, swap

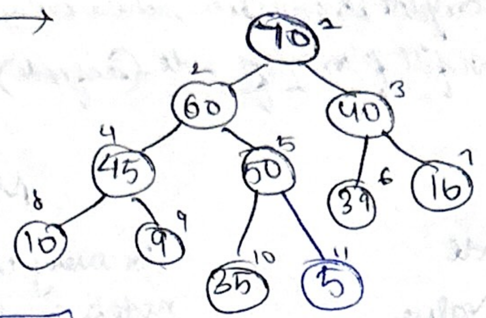


50 < 60 → again swap



70 > 60, no swap

Q Insert 5 →



$i = 11 \rightarrow \frac{i}{2} = 5$

Satisfies Max Heap, no swap

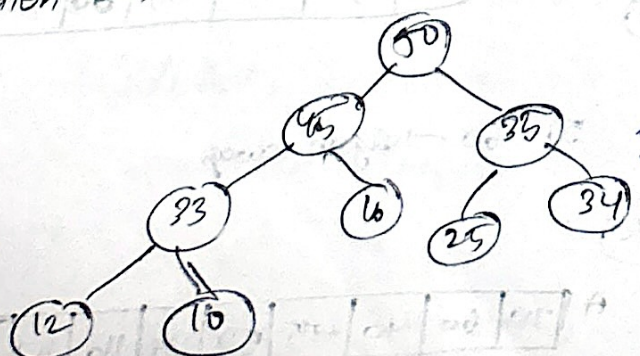
$\frac{i}{2} > i$ → No swap

∴ Time Complexity (Insertion) = $O(\log n)$ [Height of Tree]

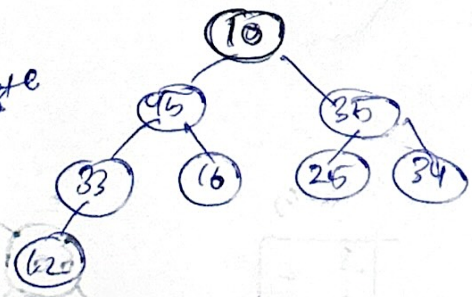
Algo → (A: Array, n: size of array)

```
InsertHeap(A, n, value) {
    n = n+1 → ek insert kara to size badhaya.
    A[n] = value;
    i = n;
    while (i>1) {
        Parent = [i/2];
        if (A[Parent] < A[i]) {
            swap(A[Parent], A[i]);
            ~~i=Parent;~~
            i = Parent;
        }
        else { return; }
    }
}
```

Deletion →



Delete 50

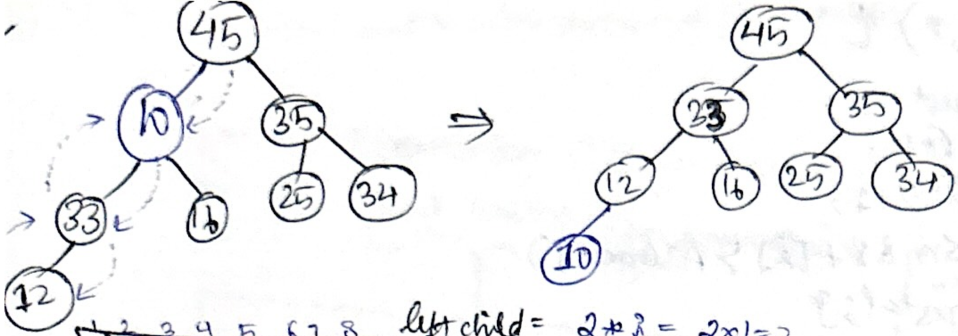


Now Heapify

Balance / Maintain Heap

After deletion

	1	2	3	4	5	6	7	8
A	10	45	35	33	16	25	34	12



1	2	3	4	5	6	7	8
10	45	35	33	16	25	34	12

i=1 → 45, 10 → 33, 10 → 12, 10

left child = 2 * i = 2×1 = 2

Right child = (2 * i) + 1 = 2×1+1 = 3

[∵ i = 1]

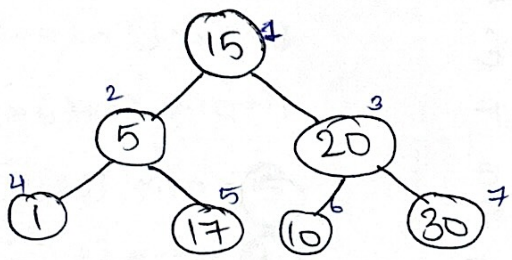
10 < 45 & 10 < 35 ⟶ swap with 45 largest

i=2: lc = 2*2 = 4, RC = (2*2)+1 = 5 } 10 < 33 & 10 < 16, swap with largest

i=4: lc = 8, Rc = 9 } 10 < 12 Yes, swap

↓ Jab tak 10 jaayega 10 index

Max Heapify ⟶

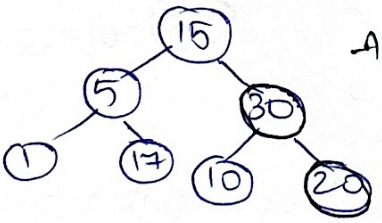


		start				
15	5	20	1	17	10	30

Leaf nodes ⇒ $\lfloor \frac{n}{2} \rfloor + 1$ to n

4 to 7

Start Heapify from Non-leaf largest index



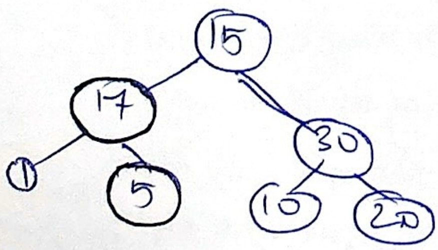
A

15	5 (17)	30	1	17 (5)	10	20

decrement, Again decrement

O(n)

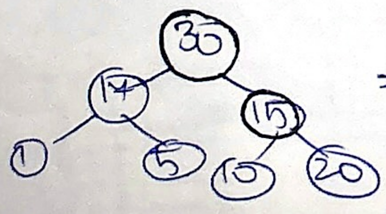
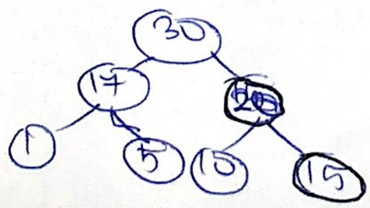
⇓



A

30	17	20	1	5	10	15

⇓

 ⇒ 

```
Max-Heapify(A, n, i) {
    int largest = i;
    int l = 2*i;
    int r = (2*i)+1;
    while (l <= n && A[l] > A[largest])
    {   largest = l; }
    while (r <= n && A[r] > A[largest]).
    {   largest = r; }
    if (largest != i) {
            swap (A[largest, A[i]);
            Heapify(A, n, largest); }
}
```

(A) Hashing → Used to store and search data quickly using a hash function.

① Open Hashing (Closed Addressing) → Chaining

② Closed Hashing (Open Addressing)

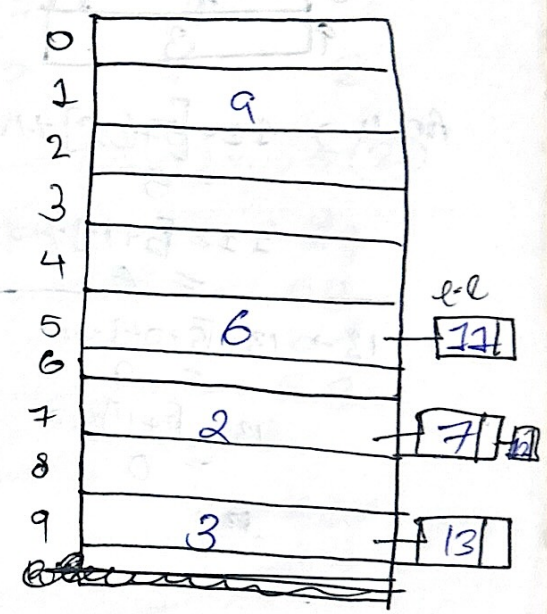
- Linear Probing
- Quadratic Probing
- Double Hashing

[hash func – h(key) = key % size]

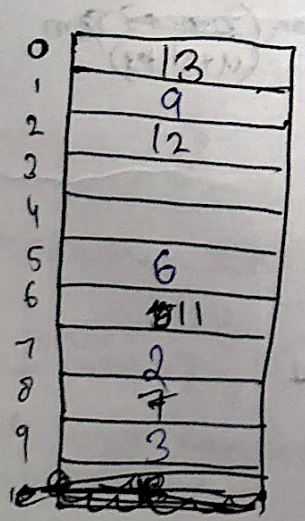
Q Use division method & closed add to store these values — A = 3, 2, 9, 6, 11, 13, 7, 12

h(k) = 2k+3 & m = 10 → Separable Chaining ⇒ Same index par linked list

key	location (k % m)
3	[(2×3)+3] % 10 = 9
2	[(2×2)+3] % 10 = 7
9	[(2×9)+3] % 10 = 1
6	[(2×6)+3] % 10 = 5
11	[(2×11)+3] % 10 = 5
13	[(2×13)+3] % 10 = 9
7	[(2×7)+3] % 10 = 7
12	[(2×12)+3] % 10 = 7



Q Above ques by Linear Probing → If index full check next index (linearly)



5 par aaya 11 again so go to index/location 6

9 par aaya 13 so go to 0

7 par aaya 7 so go to 8.

7 par aaya 12 go to 2

⇒ 13, 9, 12, –, –, 6, 11, 2, 7, 3 → Kitne baar search kara

key	location	Probes
3	9	1
2	7	1
9	1	1
6	5	1
11	6	2
13	0	2

key	
7	2
12	6

(A) Quadratic Probing ——→ Insert K_i at first free location from $(u+i^2)\%m$ where $i=0$ to $(m-1)$

u = location, $i = 0,1,2,3,4$

A = 3, 2, 9, 6, 11, 13, 7, 12

$h(k) = 2k+3$ & $m=10$

0	13
1	9
2	
3	12
4	
5	6
6	11
7	2
8	7
9	3

→ Connection @ 11

Key	Location	Probe
3	$[(2\times3)+3]\%10 = 9$	1
2	7	1
9	1	1
6	5	1
11	5	2
13	9	2
7	7	2
12	7	5 (0,1,2,3,4)

for 11 → $11 = [5+(0)^2]\%10 = 5$

$11 = [5+1^2]\%10 = 6$

13 → $13 = [9+0^2]\%10 = 9$

$13 = [9+1^2]\%10 = 0$

7 → $7 = (7+0^2)\%10 = 7$

$7 = (7+1^2)\%10 = 8$

12 → $12 = (7+1^2)\%10 = 8$ | $(7+2^2)\%10 = 1$

$(7+3^2)\%10 = 6$

$(7+4^2)\%10 = 3$

∴ 13, 9, –, 12, –, 6, 11, 2, 7, 3

(B) Double Hashing ——→ Insert K_i at first free place from $(u+v*i)\%m$

where $v = [h_2(k)\%m]$

Q A = 3, 2, 9, 6, 11, 13, 7, 12

$h_1(k) = 2k+3$

$m=10$

$h_2(k) = 3k+1$

0	
1	9
2	
3	
4	11
5	6
6	
7	2
8	
9	3

Key	Location (u)	V	Probe
3	[2(3)+3] % 10 = 9	—	1
2	[2(2)+3] % 10 = 7	—	1
9	[2(9)+3] % 10 = 1	—	1
6	[2(6)+3] % 10 = 5	—	1
11	5	4	3
13	9	9	0
7	7	2	
12	7	7	2

V for key = 11

→ [3(11)+1] % 10 = (4)

(u + v * i) % m

⇒ (5 + 4 * 0) % 10 ⇒ 5

⇒ (5 + 4 * 1) % 10 → 9

⇒ (5 + 4 * 2) % 10 → 3

V for key 7 ⇒ [3(7)+1] % 10 ⇒ (2)

(7 + 2 * 0) % 10 → 7

(7 + 2 * 1) % 10 → 9

(7 + 2 * 2) % 10 → 1

(7 + 2 * 3) % 10 → 3

(7 + 2 * 5) % 10 → 7

Till 9 it gives Rep values

V for key 13 ⇒ [3(13)+1] % 10 = (0)

(9 + 0 * 0) % 10 → 9

(9 + 0 * 1) % 10 → 9

} So, cannot insert

Cannot Insert 13

(*) Load factor → Tells Hash Table how filled has

$\alpha = \frac{n}{m}$ — n → no. of elements, m → size of hash table

Eg: $\frac{7}{10} = 0.7$ / 70% table filled.

Maintain low factor ⇒ If load factor becomes high = more collisions

How to maintain? — Then ⇒ Searching slow, Insertion slow } worst case = O(n)

1. Create larger Hash Tables
2. Recalculate Indexes
3. Insert all elements